

Adaptive Wi-Fi Bandwidth Allocation Using Machine Learning: A Comparative Study of Decision Tree and Random Forest Algorithms

Gurukiran B H

*Dept. of Electronics and Communication Engg.
R V College of Engineering
Bengaluru, Karnataka, India
gurukiranbh.ec24@rvce.edu.in*

Nishaanth K S

*Dept. of Electronics and Communication Engg.
R V College of Engineering
Bengaluru, Karnataka, India
nishaanthks.ec23@rvce.edu.in*

Abstract—Wireless local area networks increasingly serve a heterogeneous mix of devices whose traffic demands fluctuate continuously, yet most deployed access points still allocate bandwidth using static, rule-based policies that cannot react to changing channel conditions. This paper presents an adaptive bandwidth allocation framework that predicts a client-level network priority from three quality-of-service indicators, namely delay, packet loss, and throughput, and translates the predicted priority into a bandwidth recommendation. Because no public dataset labels Wi-Fi sessions by required priority, a synthetic dataset was generated by sampling the three indicators across realistic operating ranges and computing a composite priority score, which was subsequently discretised into three priority classes. Decision Tree and Random Forest classifiers were trained on an 80/20 split of the generated data and compared using accuracy, precision, recall, F1 score, and five-fold cross-validation. The Random Forest classifier achieved a test accuracy of 94.60%, exceeding the Decision Tree baseline of 89.40% by 5.20 percentage points, and showed a smaller train-test accuracy gap across increasing tree depth, indicating reduced overfitting. Feature importance analysis places delay as the most influential predictor for the Random Forest model, followed closely by packet loss and throughput. An interactive Streamlit dashboard was developed to visualise model comparisons and to issue live priority predictions and bandwidth recommendations from user-supplied network readings. The results indicate that ensemble tree-based learning offers a practical basis for adaptive, data-driven bandwidth management in Wi-Fi networks.

Index Terms—Adaptive bandwidth allocation, Wi-Fi quality of service, Decision Tree, Random Forest, network priority prediction, Streamlit dashboard, machine learning

I. INTRODUCTION

The proliferation of Wi-Fi-connected devices, spanning smartphones, laptops, Internet of Things sensors, and cloud-

dependent applications, has placed sustained pressure on wireless access points to distribute a limited and shared radio resource fairly and efficiently. As the number of simultaneously connected clients grows, contention for airtime increases, and the resulting congestion manifests as elevated delay, increased packet loss, and diminished throughput for individual sessions. These symptoms directly degrade the perceived quality of service for latency-sensitive applications such as video conferencing, online gaming, and voice communication.

Conventional bandwidth allocation mechanisms typically rely on static or semi-static policies, such as fixed per-client caps or simple round-robin scheduling, that do not respond to the instantaneous condition of the network. Because such policies are not sensitive to real-time congestion signals, they tend to under-allocate bandwidth to clients experiencing genuine degradation while over-allocating to clients whose sessions are already performing acceptably. This mismatch becomes more pronounced as network load fluctuates throughout the day, particularly in environments with bursty or unpredictable traffic patterns.

This work proposes an adaptive alternative in which bandwidth allocation decisions are driven by a machine-learning classifier that continuously infers a client's network priority from three quality-of-service indicators: delay, packet loss, and throughput. Rather than applying a fixed threshold rule to each indicator independently, the classifier learns a joint decision boundary over the three-dimensional feature space, allowing it to capture interactions between the indicators that a single-threshold rule would miss. The predicted priority, categorised as High, Medium, or Low, is then mapped to a corresponding bandwidth allocation recommendation.

Because no publicly available dataset labels Wi-Fi client sessions according to the priority levels required for this task, a synthetic dataset was constructed by sampling the three indicators over realistic operating ranges and assigning a priority label using a composite scoring function. Two supervised classifiers, Decision Tree and Random Forest, were trained on this dataset and compared in terms of predictive accuracy, generalisation behaviour, and sensitivity to model complexity. An interactive Streamlit dashboard was additionally developed

to allow live entry of network readings and to visualise model performance, feature importance, and bandwidth recommendations.

The remainder of this paper is organised as follows. Section II reviews related work on Wi-Fi resource allocation and machine-learning-based network management. Section III states the problem addressed. Section IV describes the proposed system and its architecture. Section V details dataset generation and the priority scoring methodology. Section VI presents the machine learning framework. Section VII describes the software implementation, including the dashboard. Section VIII outlines the experimental setup, and Section IX presents results and discussion. Advantages, limitations, and future scope are discussed in Sections X, XI, and XII, followed by the conclusion in Section XIII.

II. LITERATURE REVIEW

Bandwidth management in wireless local area networks has traditionally been approached through static quality-of-service configurations defined by network administrators, such as fixed traffic classes mapped to IEEE 802.11e access categories. These configurations improve differentiation between traffic types but do not adapt to instantaneous network conditions, since the mapping between traffic class and priority is decided in advance rather than inferred from live measurements.

Research on machine-learning-based network resource management has explored supervised classification for tasks such as traffic classification, congestion prediction, and admission control. Classifiers trained on flow-level statistics have been shown to distinguish between traffic types with reasonable accuracy, motivating their extension to priority-driven resource allocation problems where the objective is not to identify the application generating traffic but to infer how urgently a session requires additional resources.

Comparative studies of tree-based classifiers report that ensemble methods such as Random Forest consistently outperform single Decision Tree models on tabular, low-dimensional feature sets, primarily due to the variance-reduction effect of bootstrap aggregation. This behaviour is particularly relevant to quality-of-service prediction tasks, where the feature space is often limited to a small number of continuously valued network indicators, similar to the three-parameter feature space adopted in this work.

Software-defined networking literature has proposed centralised controllers capable of dynamically reallocating bandwidth based on real-time telemetry, demonstrating that programmable control planes can implement adaptive policies more flexibly than fixed-function access point firmware. These architectures provide a natural deployment target for a machine-learning-based priority classifier, since the controller can enforce the bandwidth recommendation produced by the model without requiring changes to client-side software.

Dashboard-based visualisation tools have become common in applied machine learning projects for presenting model comparisons, feature importance, and live predictions to non-specialist users. Frameworks such as Streamlit allow rapid

development of interactive interfaces directly from Python model code, reducing the engineering overhead of exposing a trained classifier to an operational user.

Taken together, the surveyed literature supports three observations that motivate the present work: static bandwidth policies are insufficiently responsive to real-time network conditions, ensemble tree-based classifiers are well suited to low-dimensional quality-of-service feature spaces, and interactive dashboards provide a practical mechanism for surfacing model predictions to network operators. The gap addressed in this paper is the absence of a lightweight, three-parameter priority classification framework that couples a trained ensemble model directly to a bandwidth recommendation and an accessible visualisation interface.

III. PROBLEM STATEMENT

Existing Wi-Fi bandwidth allocation mechanisms generally depend on static configuration rules that do not account for the current condition of the network as experienced by individual clients. Because delay, packet loss, and throughput vary continuously with network load, a fixed allocation policy cannot simultaneously serve clients experiencing genuine congestion and clients whose connections remain healthy without either over-provisioning or under-provisioning bandwidth for at least a portion of connected devices.

The objective of this work is to design a system that monitors the three quality-of-service indicators for each client, predicts a discrete priority level reflecting the severity of network degradation experienced by that client, and recommends a bandwidth allocation proportional to the predicted priority. The system must additionally provide a comparative evaluation of candidate classification models so that the most reliable predictor can be selected for deployment, and must expose its predictions through an interface suitable for real-time operator use.

IV. PROPOSED SYSTEM AND ARCHITECTURE

The proposed framework couples a data-driven priority classifier with a rule-based bandwidth mapping stage. Network parameters describing each client session, namely delay, packet loss, and throughput, form the input to the system. These parameters are first used during an offline stage to generate a labelled dataset and train the classification models; during the operational stage, the same three parameters are supplied either from live monitoring or through manual entry on the dashboard, and the trained classifier outputs a predicted priority level.

The system architecture proceeds through the following stages, illustrated conceptually in Fig. 1: network parameters are collected, a synthetic dataset is generated for training purposes, a composite priority score is computed and discretised into class labels, the dataset is partitioned into training and testing subsets, Decision Tree and Random Forest models are trained and evaluated, the better-performing model is used to generate priority predictions, predicted priorities are mapped

to bandwidth allocation recommendations, and results are surfaced through the Streamlit dashboard.

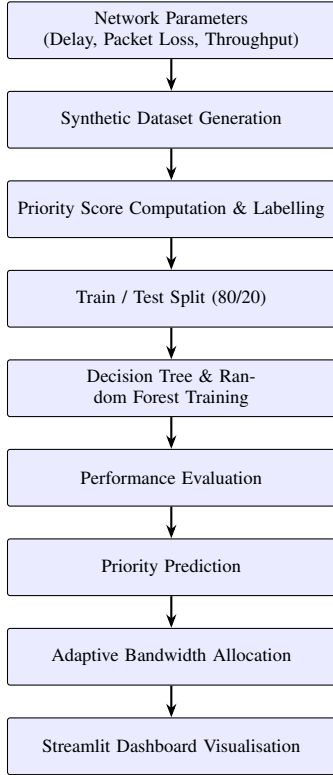


Fig. 1. Conceptual system architecture of the adaptive Wi-Fi bandwidth allocation framework, from network parameter acquisition through to dashboard-based bandwidth recommendation.

This architecture separates offline model training from on-line inference, allowing the classifier to be retrained periodically as network conditions evolve without disrupting the live prediction and allocation pipeline. The modular structure also allows the bandwidth mapping stage to be reconfigured independently of the classifier, for instance to adjust the bandwidth allotted to each priority tier without retraining the underlying model.

V. DATASET GENERATION AND PRIORITY SCORING METHODOLOGY

Because no public dataset labels Wi-Fi client sessions with the priority categories required for this task, a synthetic dataset was constructed by sampling the three input parameters across realistic operating ranges. Table I summarises the sampling ranges used for dataset generation.

TABLE I
QUALITY-OF-SERVICE PARAMETER RANGES USED FOR DATASET GENERATION

Parameter	Range	Unit
Delay	1 – 200	ms
Packet Loss	0 – 20	%
Throughput	1 – 100	Mbps

For every generated record, delay, packet loss, and throughput were sampled independently within the ranges given in Table I, and a composite priority score was computed according to (1).

$$S = \frac{D}{200} + \frac{L}{20} - \frac{T}{100} \quad (1)$$

where S denotes the priority score, D denotes delay in milliseconds, L denotes packet loss as a percentage, and T denotes throughput in megabits per second. Each term is normalised by the corresponding parameter range before combination, ensuring that delay, packet loss, and throughput contribute on a comparable numerical scale despite differing units. Delay and packet loss enter the score positively, since higher values of either indicate a more degraded connection requiring greater priority, whereas throughput enters negatively, since higher throughput indicates a healthier connection requiring less priority intervention.

The continuous score produced by (1) was discretised into three priority classes, described in Table II, using threshold boundaries selected so that each class receives a meaningful share of the generated records while preserving a clear ordinal relationship between score magnitude and priority.

TABLE II
PRIORITY CLASSES AND INTERPRETATION

Class	Interpretation
High Priority	Poor network conditions; elevated delay and packet loss combined with low throughput; requires immediate bandwidth allocation.
Medium Priority	Average network performance; moderate bandwidth allocation recommended.
Low Priority	Good network conditions; low delay and packet loss combined with high throughput; lower bandwidth requirement.

Algorithm 1 summarises the dataset generation and labelling procedure used to produce the training corpus.

Since the three input features are generated independently and combined through a fixed linear scoring function, the resulting dataset preserves a smooth relationship between the feature space and the assigned priority label, while the discretisation step introduces class boundaries that a classifier must learn to approximate through recursive partitioning of the feature space. No additional feature scaling was applied prior to model training, since both Decision Tree and Random Forest classifiers partition the feature space using threshold comparisons on individual features and are not sensitive to differences in feature magnitude.

VI. MACHINE LEARNING FRAMEWORK

Two supervised classification algorithms were trained and compared for the priority prediction task: Decision Tree and Random Forest. The Decision Tree classifier constructs a single hierarchical structure of threshold-based splits over the three input features, recursively partitioning the training data to maximise class purity at each node. Its principal advantages

Algorithm 1 Synthetic QoS Dataset Generation

Require: Record count N , parameter ranges for D , L , T , score thresholds θ_1 , θ_2

Ensure: Labelled dataset (X, y)

```
1:  $\mathcal{D} \leftarrow \emptyset$ 
2: for  $i = 1$  to  $N$  do
3:   Sample  $D_i \sim \text{Uniform}(1, 200)$ 
4:   Sample  $L_i \sim \text{Uniform}(0, 20)$ 
5:   Sample  $T_i \sim \text{Uniform}(1, 100)$ 
6:   Compute  $S_i \leftarrow \frac{D_i}{200} + \frac{L_i}{20} - \frac{T_i}{100}$ 
7:   if  $S_i \geq \theta_1$  then
8:      $y_i \leftarrow \text{High Priority}$ 
9:   else if  $S_i \geq \theta_2$  then
10:     $y_i \leftarrow \text{Medium Priority}$ 
11:   else
12:     $y_i \leftarrow \text{Low Priority}$ 
13:   end if
14:    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(D_i, L_i, T_i, y_i)\}$ 
15: end for
16: return  $(X, y)$  derived from  $\mathcal{D}$ 
```

are interpretability and low inference cost, since a prediction requires only a sequence of threshold comparisons along a single path of the tree; however, an unconstrained single tree is prone to overfitting the specific noise present in the training partition, particularly near class boundaries where the priority score changes gradually rather than sharply.

The Random Forest classifier addresses this weakness by training an ensemble of decision trees on independently bootstrap-sampled subsets of the training data, with each tree additionally restricted to a random subset of features at every split. Final predictions are obtained through majority voting across all trees in the ensemble. This combination of bootstrap aggregation and feature subsampling decorrelates the individual trees, reducing the variance of the overall prediction and typically improving generalisation relative to a single decision tree, at the cost of increased training time and reduced interpretability of any single decision path.

Both models were trained on an 80/20 split of the generated dataset, with the training partition used exclusively for model fitting and the testing partition reserved for performance evaluation. Model performance was quantified using accuracy, precision, recall, and F1 score, computed according to the standard formulations in (2)–(5), and further validated using five-fold cross-validation to assess the stability of the reported accuracy figures across different partitions of the training data.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

where TP , TN , FP , and FN denote true positive, true negative, false positive, and false negative counts respectively, computed on a per-class basis for the three-class setting.

VII. SOFTWARE IMPLEMENTATION

The complete pipeline, comprising dataset generation, priority scoring, model training, evaluation, and prediction export, was implemented in Python. Model training and evaluation relied on the Scikit-Learn library, while data manipulation was performed using Pandas and NumPy. An interactive dashboard was developed using Streamlit to expose the trained models to an end user without requiring direct interaction with the underlying Python scripts.

The dashboard provides a network overview panel summarising dataset statistics, a head-to-head model comparison view presenting test-set and cross-validation accuracy for both classifiers side by side, a feature importance view, a confusion matrix view, and a live prediction panel. In the live prediction panel, a user enters delay, packet loss, and throughput values directly, and the dashboard returns the predicted priority level from both the Decision Tree and Random Forest models along with the associated class probability distribution, together with a recommended bandwidth allocation corresponding to the predicted priority tier.

VIII. EXPERIMENTAL SETUP

All experiments were implemented in Python and executed on a standard desktop environment. The synthetic dataset was generated with a fixed random seed to ensure reproducibility of the reported results, and the same 80/20 train-test partition was used for both classifiers so that observed differences in performance reflect differences in model behaviour rather than differences in data exposure. Five-fold cross-validation was additionally performed on the training partition to assess the variance of each model's accuracy across different data folds.

Evaluation metrics recorded for each model included overall test accuracy, mean cross-validation accuracy with standard deviation, per-class precision, recall, and F1 score, and feature importance scores extracted from the trained models. An overfitting analysis was conducted by varying the maximum permitted depth of the Decision Tree and observing the resulting train and test accuracy, and a corresponding analysis was performed for the Random Forest classifier by varying the number of constituent trees.

IX. RESULTS AND DISCUSSION

A. Overall Accuracy Comparison

Fig. 2 presents the head-to-head comparison of test-set accuracy and five-fold cross-validation accuracy for the two evaluated classifiers. The Random Forest classifier achieved a test accuracy of 0.9460, compared to 0.8940 for the Decision Tree classifier, representing an improvement of 0.0520. On five-fold cross-validation, Random Forest achieved a mean

accuracy of 0.9290 with a standard deviation of 0.0174, compared to 0.8710 with a standard deviation of 0.0115 for the Decision Tree classifier.



Fig. 2. Head-to-head comparison of Decision Tree and Random Forest test-set accuracy and five-fold cross-validation accuracy.

The improvement in test accuracy is consistent with the improvement observed under cross-validation, indicating that the advantage of the Random Forest classifier is not confined to a single train-test split but persists across different partitions of the training data. The wider standard deviation associated with the Random Forest cross-validation score, relative to the Decision Tree, indicates a marginally greater sensitivity to which folds are used for training, although the mean accuracy remains consistently higher across all folds.

B. Per-Class Precision, Recall, and F1 Score

Fig. 3 shows precision, recall, and F1 score for each of the three priority classes, computed separately for the Decision Tree and Random Forest models. For the Decision Tree classifier, the Medium priority class shows the lowest scores across all three metrics, indicating that boundary cases between High and Low priority are more difficult for a single tree to separate cleanly. For the Random Forest classifier, per-class scores are more evenly distributed across High, Medium, and Low priority, with the Medium class again showing marginally lower scores than the High and Low classes, though the gap is smaller than for the Decision Tree.



Fig. 3. Per-class precision, recall, and F1 score for the Decision Tree and Random Forest classifiers.

This pattern is consistent with the construction of the priority score in (1), since the Medium priority class occupies the central portion of the continuous score distribution and

is therefore adjacent to both the High and Low decision boundaries, making it more susceptible to misclassification than the two extremal classes.

C. Feature Importance

Fig. 4 presents feature importance scores extracted from both trained models. For the Decision Tree classifier, packet loss carries the highest importance at 0.354, followed by delay at 0.340 and throughput at 0.306. For the Random Forest classifier, delay carries the highest importance at 0.362, followed by packet loss at 0.327 and throughput at 0.312.

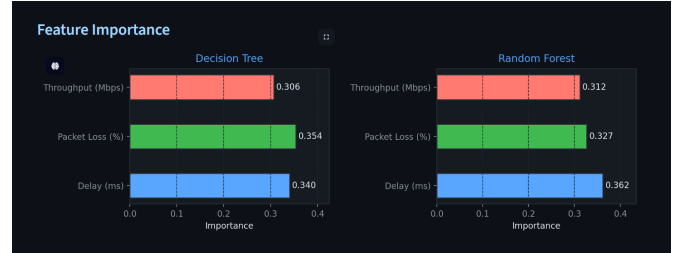


Fig. 4. Feature importance scores for delay, packet loss, and throughput extracted from the Decision Tree and Random Forest classifiers.

Although the relative ranking of the three features differs slightly between the two models, all three features contribute comparably to the overall classification decision in both cases, with importance scores for the least influential feature never falling far below those of the most influential feature. This near-uniform distribution of importance is consistent with the additive structure of (1), in which delay, packet loss, and throughput each contribute a comparably scaled term to the underlying priority score.

D. Error Analysis

Fig. 5 presents scatter plots of delay against throughput for the test partition, with correctly classified samples and misclassified samples distinguished by marker style, for both the Decision Tree and Random Forest classifiers. For both models, misclassified samples are concentrated in regions of the feature space where delay and throughput values would place the record close to a class boundary, rather than being distributed uniformly across the plot. The Random Forest classifier exhibits visibly fewer misclassified points than the Decision Tree classifier over the same test partition, consistent with the higher overall test accuracy reported in Fig. 2.

E. Overfitting Analysis

Fig. 6 shows training and test accuracy for the Decision Tree and Random Forest classifiers as a function of maximum tree depth. For the Decision Tree classifier, training accuracy continues to rise toward 1.0 as depth increases, while test accuracy plateaus at approximately 0.90 beyond a depth of roughly eight, producing a visibly widening gap between training and test accuracy that is characteristic of overfitting. For the Random Forest classifier, both training and test accuracy also rise with depth, but the gap between the two curves



Fig. 5. Error analysis scatter plots of delay against throughput for the Decision Tree and Random Forest classifiers, distinguishing correctly classified samples from misclassified samples.

remains narrower, and test accuracy plateaus at a higher level of approximately 0.94.

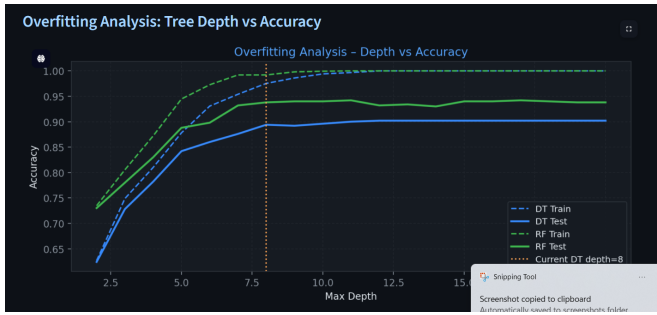


Fig. 6. Training and test accuracy of the Decision Tree and Random Forest classifiers as a function of maximum tree depth, illustrating the overfitting behaviour of the single-tree model relative to the ensemble.

This behaviour confirms that the accuracy advantage of Random Forest over Decision Tree observed in Fig. 2 is attributable, at least in part, to the ensemble's resistance to overfitting rather than solely to a difference in representational capacity, since both models are permitted to reach comparable depth without the Random Forest test accuracy declining in the way that would be expected from an overfit single tree.

F. Effect of Ensemble Size on Random Forest Accuracy

Fig. 7 shows Random Forest test accuracy as a function of the number of constituent trees. Accuracy rises sharply as the number of trees increases from a small value toward approximately 50, after which further increases in tree count yield only marginal additional improvement, with accuracy fluctuating narrowly between approximately 0.945 and 0.950 for tree counts between 100 and 200. The configuration used for the reported results, with 100 estimator trees, lies within this plateau region, indicating that the model operates close to its achievable accuracy ceiling under the current feature set without requiring an excessively large ensemble.

G. Live Prediction Example

Fig. 8 shows an example live prediction generated through the dashboard for a single set of network readings. Both classifiers predicted High Priority for the supplied input, but

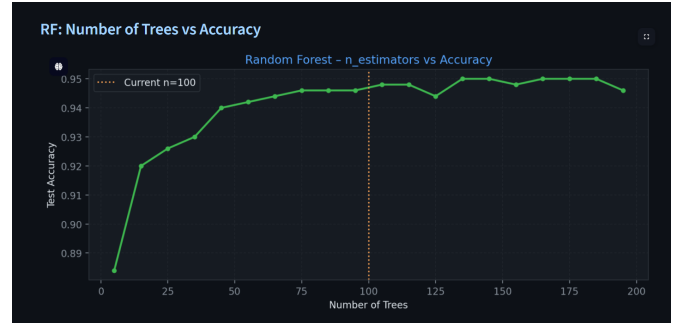


Fig. 7. Random Forest test accuracy as a function of the number of estimator trees.

the confidence distributions differ noticeably between the two models: the Decision Tree classifier assigns essentially the entire probability mass to the High Priority class, whereas the Random Forest classifier distributes a smaller but still dominant probability to High Priority, with the remaining probability mass split between Medium and, to a much smaller extent, Low Priority.

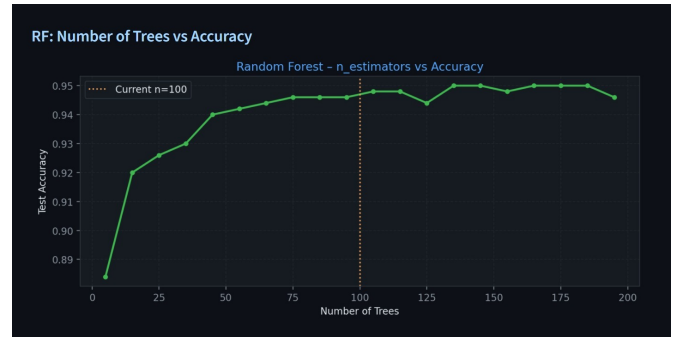


Fig. 8. Example live prediction output from the dashboard, comparing predicted class probabilities from the Decision Tree and Random Forest classifiers for a single network reading.

This difference in confidence behaviour is expected given the underlying model structures. A single decision tree assigns a prediction based on the class distribution within a single terminal leaf, which can produce near-deterministic probability outputs when the leaf is dominated by one class in the training data. Random Forest, by aggregating votes across 100 independently trained trees, produces a probability estimate that reflects the proportion of trees favouring each class, which more often results in a softer probability distribution even when the majority prediction is unambiguous. This behaviour suggests that Random Forest probability outputs may provide a more graduated and informative confidence signal for downstream bandwidth allocation decisions than the comparatively overconfident output of a single Decision Tree.

H. Bandwidth Allocation Mapping

Based on the predicted priority class, the dashboard issues a corresponding bandwidth allocation recommendation, with High Priority mapped to the largest bandwidth allotment,

Medium Priority mapped to a moderate allotment, and Low Priority mapped to the smallest allotment. Because the underlying classifier achieves consistently higher accuracy and more balanced per-class performance when implemented using Random Forest, the Random Forest model was selected as the classifier driving the bandwidth recommendation logic in the deployed dashboard, while the Decision Tree model is retained within the dashboard for comparative visualisation.

X. ADVANTAGES

The proposed framework offers several advantages over static bandwidth allocation policies. Priority is inferred directly from real-time network indicators rather than from a fixed configuration, allowing the system to adapt to changing congestion levels without manual reconfiguration. The three-feature input space keeps both training and inference computationally lightweight, making the framework suitable for near-real-time operation. The Random Forest classifier provides improved generalisation and reduced overfitting relative to a single Decision Tree, while remaining substantially simpler to train and interpret than deep learning alternatives. The accompanying Streamlit dashboard provides an accessible interface through which network operators can compare models, inspect feature importance, and obtain live predictions without requiring familiarity with the underlying Python implementation. The modular separation between the classification stage and the bandwidth mapping stage additionally allows the allocation policy to be adjusted independently of the trained model.

XI. LIMITATIONS

The dataset used for training and evaluation in this study is entirely synthetic, generated from independently sampled parameter ranges and a fixed linear scoring function rather than from measurements collected on a live Wi-Fi deployment. Reported accuracy figures should therefore be interpreted as a demonstration of relative model behaviour rather than a guaranteed real-world performance figure. The framework currently considers only three quality-of-service parameters, namely delay, packet loss, and throughput, and does not incorporate other indicators such as signal strength, jitter, or the number of contending devices that may also influence perceived network quality. The system does not interface directly with physical Wi-Fi router hardware or a software-defined networking controller to enforce the recommended bandwidth allocation; the current implementation produces a recommendation rather than an enforced allocation. Finally, the priority scoring function in (1) applies fixed linear weighting to the three input parameters, which may not capture more complex, non-linear relationships between network conditions and perceived quality of service that could exist in practical deployments.

XII. FUTURE SCOPE

Several directions could extend the present framework toward practical deployment. Integrating the system with live Wi-Fi routers or access point firmware would allow priority

prediction to operate on real monitored traffic rather than synthetically generated records, and would allow bandwidth recommendations to be enforced automatically rather than merely displayed. Incorporating additional network indicators, such as received signal strength, jitter, and the number of actively contending devices, could improve the granularity and accuracy of priority prediction beyond what is achievable using delay, packet loss, and throughput alone. Evaluation of additional classification models, including Gradient Boosting, XGBoost, and neural network architectures, may reveal further accuracy gains, particularly if non-linear relationships between input features and true network priority prove more pronounced than those encoded in the synthetic dataset used in this study. Deployment on software-defined networking infrastructure would allow the framework to draw on centralised telemetry and enforcement capabilities already present in such environments, and extension toward emerging fifth-generation and sixth-generation wireless standards and dense Internet of Things deployments represents a further avenue for future investigation.

XIII. CONCLUSION

This paper presented an adaptive Wi-Fi bandwidth allocation framework that predicts client network priority from delay, packet loss, and throughput, and translates the predicted priority into a bandwidth allocation recommendation. A synthetic dataset was generated using a composite priority scoring function, and Decision Tree and Random Forest classifiers were trained and compared on the resulting three-class problem. The Random Forest classifier achieved a test accuracy of 94.60%, exceeding the Decision Tree baseline of 89.40%, and demonstrated a smaller train-test accuracy gap across increasing model complexity, indicating stronger resistance to overfitting. Feature importance analysis indicated that delay, packet loss, and throughput each contribute comparably to the classification decision, consistent with the additive structure of the underlying priority score. An interactive Streamlit dashboard was developed to present model comparisons, feature importance, and live priority predictions, together with corresponding bandwidth allocation recommendations. The results indicate that ensemble tree-based classification provides a practical and computationally lightweight foundation for adaptive, data-driven bandwidth management, offering a basis that future work could extend toward integration with live network hardware and additional quality-of-service indicators.

REFERENCES

- [1] IEEE Standard for Information Technology–Telecommunications and Information Exchange between Systems–Local and Metropolitan Area Networks–Specific Requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications Amendment: Enhancements for Quality of Service, IEEE Std 802.11e, 2005.
- [2] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [4] N. McKeown *et al.*, “OpenFlow: Enabling innovation in campus networks,” *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

- [5] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] Streamlit Inc., “Streamlit documentation,” [Online]. Available: <https://docs.streamlit.io>. [Placeholder reference; access date to be confirmed by author.]
- [7] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.